Guion de la presentación final

Sistema web para análisis inteligente de imágenes en AWS Cloud Computing, Universidad del Valle de Guatemala, Campus Altiplano. Miércoles 11 de noviembre de 2026, 7:00 p. m. Vestimenta formal.

Este es el texto que dice cada uno, en el orden de las diapositivas. No hace falta memorizarlo palabra por palabra: la idea es que lo lean varias veces, lo digan con sus propias palabras y entiendan el porqué de cada decisión. La presentación dura unos 18 minutos, incluida la demostración.

El catedrático puede preguntarle a cualquiera sobre cualquier parte del sistema. Por eso, aunque cada quien tiene su turno, conviene que todos lean el guion completo. Al final hay respuestas preparadas para las preguntas más probables.

Diapositiva 1: Portada (Cristina)

> Buenas noches, ingeniero. Somos el equipo del proyecto final de Cloud Computing y vamos a presentar nuestro sistema web para análisis inteligente de imágenes en AWS.
>
> En pocas palabras, construimos una página web donde una persona sube una fotografía y el sistema le dice qué contiene, usando inteligencia artificial de Amazon. Lo más importante para nosotros no fue solo que funcionara, sino que siguiera funcionando aunque uno de sus servidores fallara. Eso es lo que vamos a demostrar en vivo.

Diapositiva 2: Equipo (Cristina, y cada integrante dice su parte)

> Cada integrante administró un componente del sistema. Les presento al equipo.

Cada quien dice su línea:

> **Cristina:** Yo me encargué de la red, los grupos de seguridad y el balanceador de carga de Amazon.
>
> **Isidro:** Yo configuré los dos balanceadores HAProxy y las comprobaciones de estado.
>
> **Jorge:** Yo desarrollé la aplicación web y su integración con Amazon S3 y Amazon Rekognition.

Edwin: Yo administré la base de datos en Amazon RDS, los respaldos y la guía de recuperación.

Kris: Yo hice la plantilla de CloudFormation que crea toda la infraestructura, el servidor bastión, las pruebas de continuidad y la documentación.

Diapositiva 3: Problema y objetivo (Cristina)

El problema del que partimos tiene dos partes. La primera es que clasificar imágenes a mano es lento, es subjetivo y no escala: si una organización tiene miles de fotos, nadie puede etiquetarlas una por una. La segunda es que, si una aplicación vive en un solo servidor, cuando ese servidor falla, la aplicación deja de responder.

Nuestro objetivo fue construir un sistema en la nube que resuelva las dos cosas: que suba imágenes, las analice automáticamente con Amazon Rekognition, guarde el resultado en una base de datos y que siga disponible aunque falle una de sus réplicas.

Diapositiva 4: Arquitectura (Cristina e Isidro)

Cristina: Esta es la arquitectura completa. Si la leemos de arriba hacia abajo, el usuario entra por un Application Load Balancer, que es el único punto público del sistema y está repartido en dos zonas de disponibilidad de AWS.

Toda la red está dividida en capas. Los balanceadores HAProxy están en subredes públicas, pero las réplicas de la aplicación y la base de datos están en subredes privadas, es decir, no se pueden alcanzar desde internet. Además, cada capa solo acepta tráfico de la capa anterior: el balanceador solo habla con HAProxy, HAProxy solo habla con la aplicación y la aplicación es la única que puede conectarse a la base de datos.

Isidro: Debajo del balanceador de Amazon están los dos HAProxy, uno en cada zona. Cada HAProxy reparte las solicitudes entre las dos réplicas de la aplicación; por eso las flechas se cruzan. Así no existe ningún servidor que, si se cae, tumbe el sistema completo.

Abajo están los servicios administrados que usa la aplicación: S3 para guardar las imágenes, Rekognition para analizarlas, RDS como base de datos y Secrets Manager para guardar la contraseña de la base. A la izquierda está el bastión, que es el quinto servidor y lo usamos solo para administrar y hacer respaldos.

Diapositiva 5: Flujo de una imagen (Jorge)

> Ahora les explico qué pasa cuando alguien sube una imagen, en cinco pasos.
>
> Primero, la aplicación valida el archivo. No basta con que se llame ".jpg": revisamos los primeros bytes del archivo, que son como su firma, para confirmar que de verdad es una imagen JPG o PNG. También rechazamos todo lo que pese más de 5 megabytes.
>
> Segundo, la imagen se guarda en Amazon S3 con un nombre único, generado al azar, para que nadie pueda sobrescribir la imagen de otra persona.
>
> Tercero, le pedimos a Amazon Rekognition que la analice. Rekognition nos devuelve etiquetas como "naturaleza" o "cielo", cada una con un porcentaje de confianza, y nos quedamos solo con las que tienen 70 % o más.
>
> Cuarto, registramos en la base de datos el nombre de la imagen, sus etiquetas, la confianza y la fecha.
>
> Y quinto, mostramos el resultado al usuario, que también queda guardado en el historial.

Diapositiva 6: La aplicación (Jorge)

> Así se ve la aplicación. A la izquierda está la página de resultado: la imagen y, al lado, cada etiqueta con una barra que representa su nivel de confianza. Un detalle de seguridad: la imagen se muestra con un enlace temporal que dura 15 minutos, así que el bucket de S3 nunca es público.
>
> A la derecha está el historial, en forma de galería, con todas las imágenes procesadas de la más reciente a la más antigua. Agregamos un botón para borrar el historial. Antes de borrar, el sistema pide confirmación, y lo que se elimina son los registros de la base de datos. Las imágenes se quedan en S3 como respaldo, porque decidimos que la aplicación no tenga permiso de borrar archivos. Eso es aplicar el principio de privilegio mínimo.

Diapositiva 7: Tecnologías (Jorge)

> Para elegir las tecnologías seguimos una regla: usar servicios administrados de AWS donde se puede y código abierto donde aporta.
>
> La aplicación está hecha en Python con Flask, que es un framework sencillo y suficiente para nuestras cuatro páginas, y la servimos con Gunicorn. Para hablar con AWS usamos

Boto3, que es el kit oficial de Amazon para Python.

Para la base de datos elegimos RDS con PostgreSQL en lugar de DynamoDB, por tres razones: el historial se consulta ordenado por fecha, y eso es natural en SQL; RDS nos da respaldos y restauración casi sin configuración; y nos permite demostrar claramente que la base de datos está en una red privada. La contraseña la guarda Secrets Manager, y toda la infraestructura se crea con CloudFormation, como les va a explicar Kris.

Diapositiva 8: Alta disponibilidad (Isidro)

Esta es la idea central del proyecto: tenemos dos niveles de balanceo, y cada uno vigila al siguiente.

En el primer nivel, el balanceador de Amazon revisa cada cinco segundos que los dos HAProxy estén vivos. Si uno falla dos veces seguidas, lo saca de circulación y manda todo el tráfico al otro.

En el segundo nivel, cada HAProxy revisa cada cinco segundos que las réplicas de la aplicación respondan, y si una falla tres veces la marca como caída. Pero lo más importante es que configuramos reintentos: si HAProxy intenta conectarse a una réplica y no puede, reintenta en la otra. Por eso, cuando apagamos una réplica, el usuario no ve ni un solo error.

Por último, si la aplicación se cae, el sistema operativo la vuelve a levantar sola, y cuando regresa, los balanceadores la reincorporan automáticamente. Incluso las actualizaciones las hacemos con una recarga suave, que no corta a nadie que esté usando el sistema.

Diapositiva 9: Seguridad (Cristina)

Aplicamos cuatro controles de seguridad, uno por cada cosa que pedían los lineamientos.

El primero es privilegio mínimo: los servidores tienen un rol de Amazon que solo les permite lo indispensable, que es guardar y leer imágenes, analizarlas con Rekognition y leer un único secreto.

El segundo es que no hay contraseñas en el código. La contraseña de la base de datos la generó automáticamente Secrets Manager al crear el sistema; ninguno de nosotros la escribió ni la conoce, y por eso tampoco aparece en GitHub.

El tercero es que la base de datos es privada: no tiene dirección pública y solo la aplicación puede conectarse a ella.

Y el cuarto es la validación de archivos que explicó Jorge: tipo real, tamaño máximo y nombres únicos. Además, para administrar los servidores no usamos llaves de SSH, sino AWS Systems Manager, que no requiere abrir puertos adicionales.

Diapositiva 10: Infraestructura como código (Kris)

Todo lo que hemos mostrado hasta ahora no lo creamos haciendo clic en la consola. Lo escribimos en una sola plantilla de AWS CloudFormation que crea los 39 recursos del sistema: la red con sus seis subredes, los cinco servidores, el balanceador, el bucket, la base de datos, el secreto y los permisos.

Los servidores se configuran solos al encenderse: cada uno instala lo que le toca, escribe su configuración y arranca sus servicios. Eso tiene dos ventajas. La primera es que el sistema se puede recrear completo en unos veinte minutos, sin pasos manuales ni olvidos. La segunda es que, si perdemos un servidor, CloudFormation lo vuelve a crear igual que antes.

Elegimos CloudFormation en lugar de Terraform porque viene integrado en AWS y no requiere instalar nada: la plantilla se sube directamente desde la consola. Todo el código está en nuestro repositorio de GitHub, y la aplicación tiene diez pruebas automáticas.

Diapositiva 11: Demostración en vivo (Isidro, Jorge y Kris)

Isidro: Ahora vamos a demostrar todo esto en vivo, sobre el sistema real que está corriendo en AWS en este momento.

Jorge: Primero subo una imagen… Aquí está el resultado: estas son las etiquetas que encontró Rekognition, con su porcentaje de confianza. Si voy a la consola de S3, aquí está la imagen guardada con su nombre único. Y en el historial aparece de primera.

Ahora intento subir un documento PDF al que le cambié la extensión a ".jpg"… y el sistema lo rechaza, porque revisa el contenido real del archivo. Lo mismo pasa con esta imagen de más de 5 megas.

Kris: En esta terminal estoy consultando el sistema cada segundo. En cada línea ven el código de respuesta, que es 200 cuando todo está bien, cuál de los dos HAProxy

respondió y cuál de las dos réplicas atendió. Como ven, se van alternando.

Isidro: Ahora vamos a simular que falla una réplica. Kris, apaga la aplicación en app-a, por favor... Fíjense en la terminal: todas las respuestas siguen en 200, pero ahora solo responde app-b. HAProxy detectó la caída y reintentó en la otra réplica, así que el usuario no vio ningún error. Kris la vuelve a encender... y en unos segundos app-a regresa sola.

Ahora hacemos lo mismo con un balanceador, pero como se haría un mantenimiento real: primero le decimos al balanceador de Amazon que deje de enviarle tráfico a haproxy-a y después lo apagamos. El sitio sigue respondiendo por haproxy-b. Al final lo volvemos a registrar.

Jorge: Por último, presiono "Borrar historial"... El sistema me pide confirmación, acepto, y el historial queda vacío. Vuelvo a analizar una imagen para que vean que todo sigue funcionando normalmente.

Diapositiva 12: Resultados (Edwin)

Hicimos catorce pruebas sobre el sistema real en AWS y las catorce cumplieron. Les quiero resaltar tres números.

El primero es cero: cero errores cuando apagamos una réplica de la aplicación, porque HAProxy reintentó en la otra.

El segundo es nueve segundos, que es lo que tarda el balanceador de Amazon en detectar que un HAProxy se apagó de golpe. En esos nueve segundos hubo cinco errores de 41 solicitudes. Por eso, cuando hicimos el retiro ordenado, avisándole al balanceador antes de apagar, hubo cero errores de 66. Esto nos enseñó que el mantenimiento se debe hacer de forma ordenada.

Y el tercero es cero de 133: cuando actualizamos la aplicación con recarga suave, una réplica a la vez, no hubo ningún error. La primera vez la habíamos actualizado reiniciando el servicio y hubo un error de 196, así que cambiamos el procedimiento.

También cumplieron las pruebas de etiquetas, de validación de archivos, del historial y su borrado, de la consulta a la base de datos, de que la base no es pública y de que el código no tiene secretos. Todas tienen su captura en el informe.

Diapositiva 13: Recuperación y respaldos (Edwin)

> Para cada componente tenemos documentado cómo volver a la normalidad.
>
> Si se cae una réplica de la aplicación, el sistema operativo la reinicia; y si se pierde el servidor completo, CloudFormation lo vuelve a crear. Con un nodo de HAProxy pasa lo mismo: primero se valida la configuración y se reinicia, y si hace falta, se recrea.
>
> La base de datos tiene respaldos automáticos de siete días, así que podemos restaurarla al minuto exacto que necesitemos, y además tomamos un respaldo manual antes de esta presentación. Por otro lado, el bastión hace todos los días una copia lógica de los datos y la guarda en S3. Las imágenes también están protegidas, porque el bucket guarda versiones anteriores de cada archivo.
>
> Y las actualizaciones de la aplicación se hacen una réplica a la vez, con recarga suave, que ya vimos que no produce errores.

Diapositiva 14: Conclusiones (Kris)

> Para cerrar, revisamos nuestros objetivos. Construimos una aplicación web que analiza imágenes con Rekognition y conserva un historial consultable y que se puede borrar. Integramos S3, Rekognition y RDS en un solo flujo automático. Eliminamos los puntos únicos de falla en la capa de cómputo y lo demostramos con mediciones reales. Aplicamos los controles de seguridad que pedían los lineamientos. Y dejamos documentado cómo recuperar cada componente.
>
> También sabemos qué mejoraríamos. La base de datos y el NAT Gateway son hoy los componentes con menos redundancia, así que lo primero sería activar Multi-AZ en RDS y agregar un segundo NAT Gateway. Después agregaríamos escalado automático, HTTPS con un certificado y usuarios con inicio de sesión.

Diapositiva 15: Preguntas (todos)

> **Cristina:** Muchas gracias por su atención. Quedamos atentos a sus preguntas.

Respuestas preparadas para las preguntas más probables

Respondan con calma y con sus palabras. Si la pregunta es sobre la parte de otro, pueden contestar lo general y pedirle al compañero que complemente.

¿Por qué usaron un balanceador de Amazon y además HAProxy, si los dos balancean? (Isidro)

> HAProxy era obligatorio en el proyecto, y es el que reparte entre las réplicas, con comprobaciones y reintentos. Pero si solo tuviéramos HAProxy, ese HAProxy sería un punto único de falla. Por eso pusimos dos, uno en cada zona, y delante el balanceador de Amazon, que nos da una sola dirección pública y saca de circulación al HAProxy que falle.

¿Qué pasa si se cae toda una zona de AWS? (Cristina)

> El balanceador deja de enviar tráfico al HAProxy de esa zona, y el HAProxy de la otra zona sigue atendiendo con la réplica que le queda. La base de datos hoy está en una sola zona; por eso nuestra primera recomendación es activar Multi-AZ.

¿Por qué hubo errores al apagar un HAProxy y no al apagar una réplica? (Isidro)

> Porque HAProxy reintenta en la otra réplica cuando una conexión falla, pero el balanceador de Amazon no reintenta: necesita unos nueve segundos para darse cuenta de que el HAProxy cayó. La forma correcta de hacer mantenimiento es quitarlo primero del balanceador, y así lo probamos con cero errores.

¿Cómo sabe la aplicación que un archivo es realmente una imagen? (Jorge)

> Revisa los primeros bytes del archivo. Todo JPG empieza con los bytes FF D8 FF y todo PNG con 89 50 4E 47. Si el archivo dice ser ".jpg" pero no empieza así, lo rechazamos. Además, el tamaño máximo es de 5 megas.

¿Dónde está la contraseña de la base de datos? (Edwin)

> En AWS Secrets Manager. La generó automáticamente CloudFormation al crear el sistema. La aplicación la lee al iniciar usando el rol de su servidor, así que la contraseña nunca se escribió en el código ni en ningún documento.

¿Cómo demuestran que la base de datos no es pública? (Edwin)

> Su dirección resuelve a una IP privada, la 10.0.21.108, y al intentar conectarnos desde internet al puerto de PostgreSQL no hay respuesta. Además, la configuración de RDS muestra el acceso público desactivado.

¿Qué hace exactamente el botón de borrar historial? (Jorge)

> Borra todos los registros de la tabla de imágenes, y sus etiquetas se borran en cascada. Las imágenes se conservan en S3 porque la aplicación no tiene permiso de borrar archivos; eso es privilegio mínimo y además funciona como respaldo. Lo probamos en el sistema real y quedó registrado en los dos servidores.

¿Cómo actualizaron la aplicación sin apagar el sitio? (Kris)

> Una réplica a la vez. Mientras una se actualiza, la otra atiende. Y usamos una recarga suave: Gunicorn arranca los procesos con el código nuevo y deja que los anteriores terminen lo que estaban atendiendo. Así medimos cero errores en 133 solicitudes.

¿Por qué CloudFormation y no Terraform? (Kris)

> Porque CloudFormation viene integrado en AWS, no hay que instalar nada y la plantilla se sube desde la consola. Para un proyecto que vive completamente en AWS fue lo más directo. Terraform tendría sentido si usáramos varias nubes.

¿Por qué RDS y no DynamoDB? (Edwin o Jorge)

> Porque el historial se consulta ordenado por fecha y relacionando imágenes con etiquetas, y eso es natural en una base relacional. RDS además nos da respaldos y restauración a un punto en el tiempo, y nos permitió demostrar claramente el acceso privado.

¿Cuánto cuesta mantener esto? (Kris)

> Varios recursos cobran por hora, sobre todo el NAT Gateway, el balanceador y la base de datos. Por eso, después de la presentación vamos a borrar todo el stack. Como está en una

> plantilla, podemos volver a crearlo cuando queramos.

Antes de presentar

- Cada quien lee su parte en voz alta al menos dos veces, y lee las respuestas preparadas.
- Kris y Jorge dejan abiertos el sitio, la consola de S3, el target group del balanceador y CloudShell con la sesión de AWS iniciada. Los comandos de la demo están en `DESPLIEGUE.md` y se pueden copiar de ahí.
- Comprobar que el sitio abre en la computadora con la que se va a proyectar.
- Tener la presentación en PDF descargada por si falla internet.
- Si algo falla en la demo, mostrar las capturas del informe (Figuras 8 a 14): son del mismo sistema y de las mismas pruebas.
- Vestimenta formal.